

Big Data Analysis

(Análise de Dados de Grandes Dimensões)

[Apache Spark](#)

University of Minho
2025-2026



Apache Spark

- **Distributed engine for large-scale data processing.**
- Originated at UC Berkeley in 2009.
- Often used instead of MapReduce for faster iterative and interactive workloads.
- Speeds up many workloads with **in-memory computation** and **efficient execution plans.**
- **Common uses:** ETL, SQL analytics, machine learning, streaming pipelines



Apache Spark Ecosystem.

<https://www.databricks.com/sites/default/files/2023-03/largest-open-source-apache-spark.png?v=1679038543>

Apache Spark Features

- ◎ **Distributed processing framework** built for large-scale analysis.
 - ▶ In-memory computing.
 - ▶ Unified analytics.
 - ▶ Polyglot programming.
 - ▶ Built-in libraries.
 - ▶ Fault-tolerance.
 - ▶ Lazy evaluation.
 - ▶ Advanced DAG Execution.

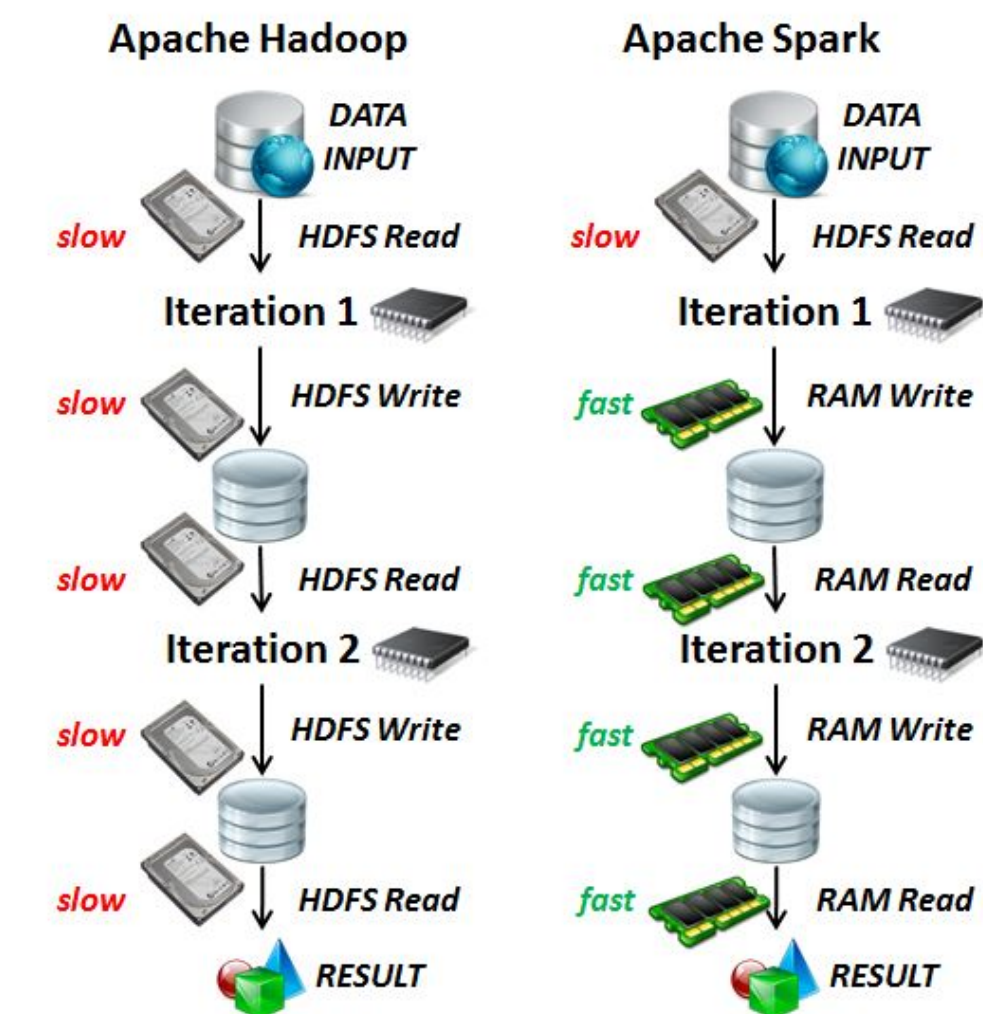
Apache Spark Features

◎ **Distributed processing framework** built for large-scale analysis.

- ▶ In-memory computing.
- ▶ Unified analytics.
- ▶ Polyglot programming.
- ▶ Built-in libraries.
- ▶ Fault-tolerance.
- ▶ Lazy evaluation.
- ▶ Advanced DAG Execution.

Fast execution. Stores intermediate data in memory, processing it faster than traditional disk-based systems.

In-memory caching + optimized execution plans



Apache Spark Features

● **Distributed processing framework** built for large-scale analysis.

- ▶ In-memory computing.
- ▶ Unified analytics.
- ▶ Polyglot programming.
- ▶ Built-in libraries.
- ▶ Fault-tolerance.
- ▶ Lazy evaluation.
- ▶ Advanced DAG Execution.



Allows batch processing, real-time streaming, machine learning, and interactive queries.

Apache Spark Features

● **Distributed processing framework** built for large-scale analysis.

- ▶ In-memory computing.
- ▶ Unified analytics.
- ▶ Polyglot programming.
- ▶ Built-in libraries.
- ▶ Fault-tolerance.
- ▶ Lazy evaluation.
- ▶ Advanced DAG Execution.



Java, Scala, Python and R.

Apache Spark Features

◎ **Distributed processing framework** built for large-scale analysis.

- ▶ In-memory computing.
- ▶ Unified analytics.
- ▶ Polyglot programming.
- ▶ Built-in libraries.
- ▶ Fault-tolerance.
- ▶ Lazy evaluation.
- ▶ Advanced DAG Execution.



SQL, MLlib, GraphX and Streaming.

Apache Spark Features

◎ **Distributed processing framework** built for large-scale analysis.

- ▶ In-memory computing.
- ▶ Unified analytics.
- ▶ Polyglot programming.
- ▶ Built-in libraries.
- ▶ Fault-tolerance.
- ▶ Lazy evaluation.
- ▶ Advanced DAG Execution.



Offered by RDDs, immutable collections of objects distributed/partitioned across a cluster.
It is based on lineage and recomputation.

Apache Spark Features

● **Distributed processing framework** built for large-scale analysis.

- ▶ In-memory computing.
- ▶ Unified analytics.
- ▶ Polyglot programming.
- ▶ Built-in libraries.
- ▶ Fault-tolerance.
- ▶ Lazy evaluation.
- ▶ Advanced DAG Execution.



The command is not executed immediately. First, it creates an execution plan and is only computed when needed.

Apache Spark Features

● **Distributed processing framework** built for large-scale analysis.

- ▶ In-memory computing.
- ▶ Unified analytics.
- ▶ Polyglot programming.
- ▶ Built-in libraries.
- ▶ Fault-tolerance.
- ▶ Lazy evaluation.
- ▶ Advanced DAG Execution.



Uses Direct Acyclic Graph (DAG) to optimize the chain of operations.

Glossary

- **Partition** - A partition is a single slice (e.g., ≤ 128 MB chunks of data) of a larger dataset.
- **Spark Job** - A parallel computation consisting of multiple tasks that gets spawned in response to a Spark action (e.g. save, collect).
- **Stages** - Each job is divided into smaller sets of tasks called stages that depend on one another, similar to the map and reduce stages in MapReduce.
- **Tasks** - A unit of work that will be sent to one executor.

Spark Architecture Overview

● Spark Core:

- Scheduling, memory management, distributed execution.

● Spark SQL:

- Structured data processing using SQL, DataFrames and an optimiser (Catalyst).

● Spark Streaming (Structured Streaming)

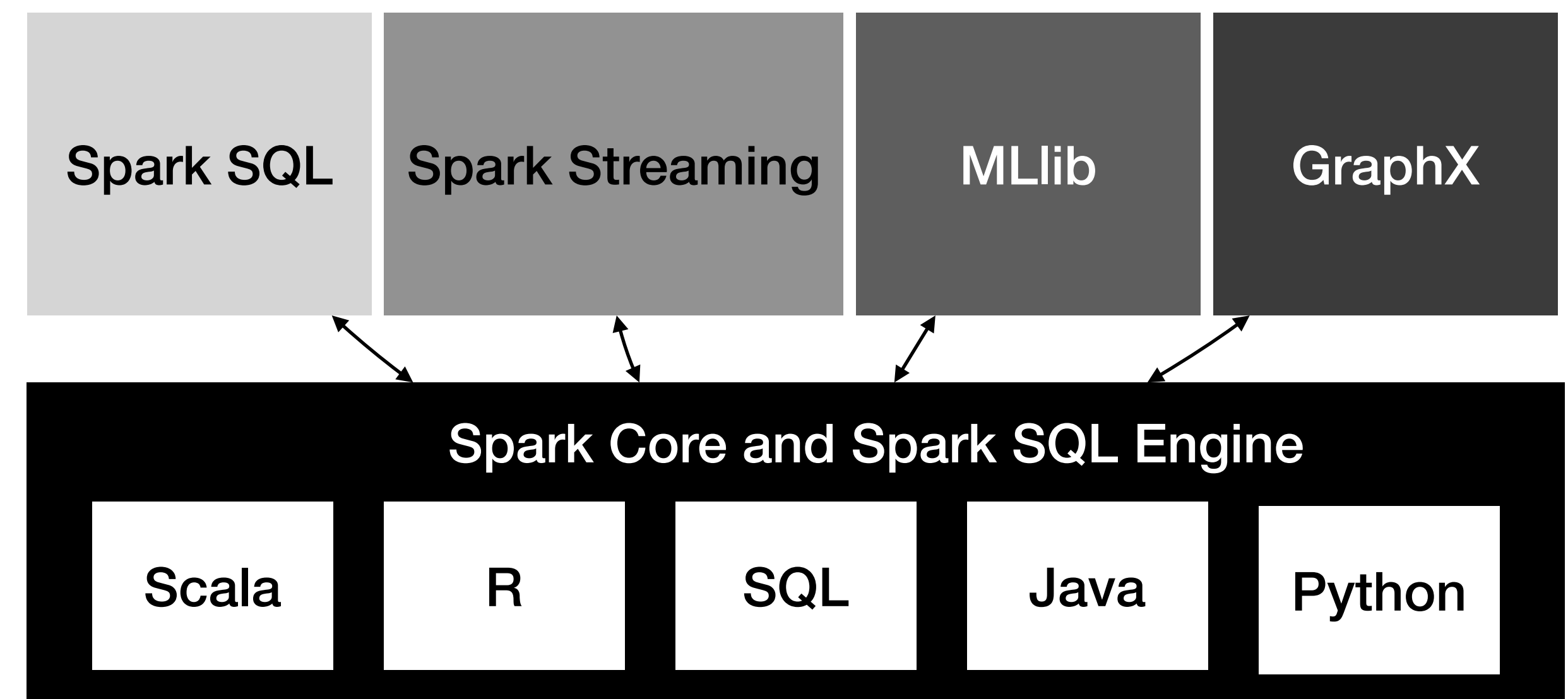
- Real-time data stream processing.

● MLlib

- Scalable ML algorithms and pipelines.

● GraphX

- Graph computation framework.



Spark Architecture Components

- **Driver Process:**

- Coordinates the application.

- **Cluster Manager:**

- Allocates resources (standalone, YARN, K8s)

- **Executors** (on worker nodes):

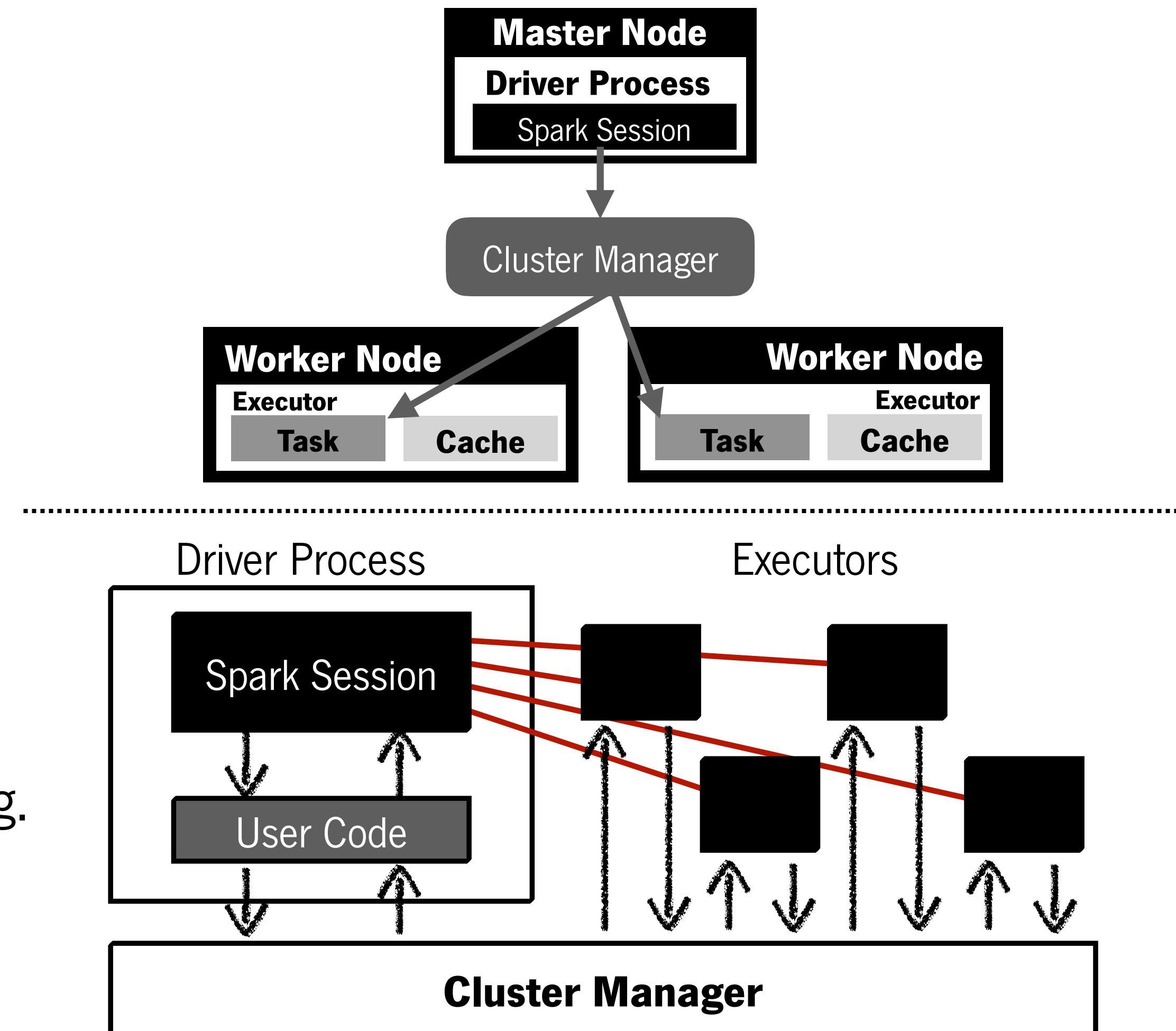
- Run the tasks and store data in memory.

- **SparkSession:**

- Entry point for DataFrames/SQL/Structured Streaming.

- **RDDs/DataFrames:**

- Distributed data abstractions processed by Spark.



Adapted from Spark: The definitive Guide (Figure 2.1)

Driver Process

- The Driver Process runs the main program and coordinates the Spark application.
- Responsibilities:
 - ❖ Create the **SparkSession** and underlying **SparkContext**.
 - ❖ Convert **user code** into a **DAG/ logical plan** for the scheduled transformations.
- Splits work into **jobs**, then **stages**, and finally **tasks** and schedules tasks on executors.
- Collects and processes the final results and handles failures/retries.

SparkSession

- **Entry point** for a Spark application.
- Connects to the **Cluster Manager**.
- Creates **RDDs**, manages DAG execution, schedules jobs and track execution.
- **Interface** between the Driver and Cluster Manager.

```
from pyspark.sql import SparkSession
import time

spark = SparkSession.builder.master("local[1]") \
    .appName('IntroToSpark') \
    .getOrCreate()
```

Cluster Manager

- Allocates CPU/memory resources for Spark applications.
- Launches executors and manages resource scheduling.
- Spark supports multiple cluster managers:
 - **Standalone Mode**
 - Built-in resource manager.
 - **YARN (Yet Another Resource Negotiator)**
 - Used in Hadoop clusters.
 - **Mesos** (currently deprecated)
 - Multi-framework cluster management.
 - **Kubernetes**
 - Containerized deployment for Spark.
- The user code does not change; the deployment configuration does.

```
$ ./bin/spark-submit --class  
org.apache.spark.examples.SparkPi \  
  --master yarn \  
  --deploy-mode cluster \  
  --driver-memory 4g \  
  --executor-memory 2g \  
  --executor-cores 1 \  
  --queue thequeue \  
examples/jars/spark-examples*.jar \  
10
```

Dynamic Resource Allocation

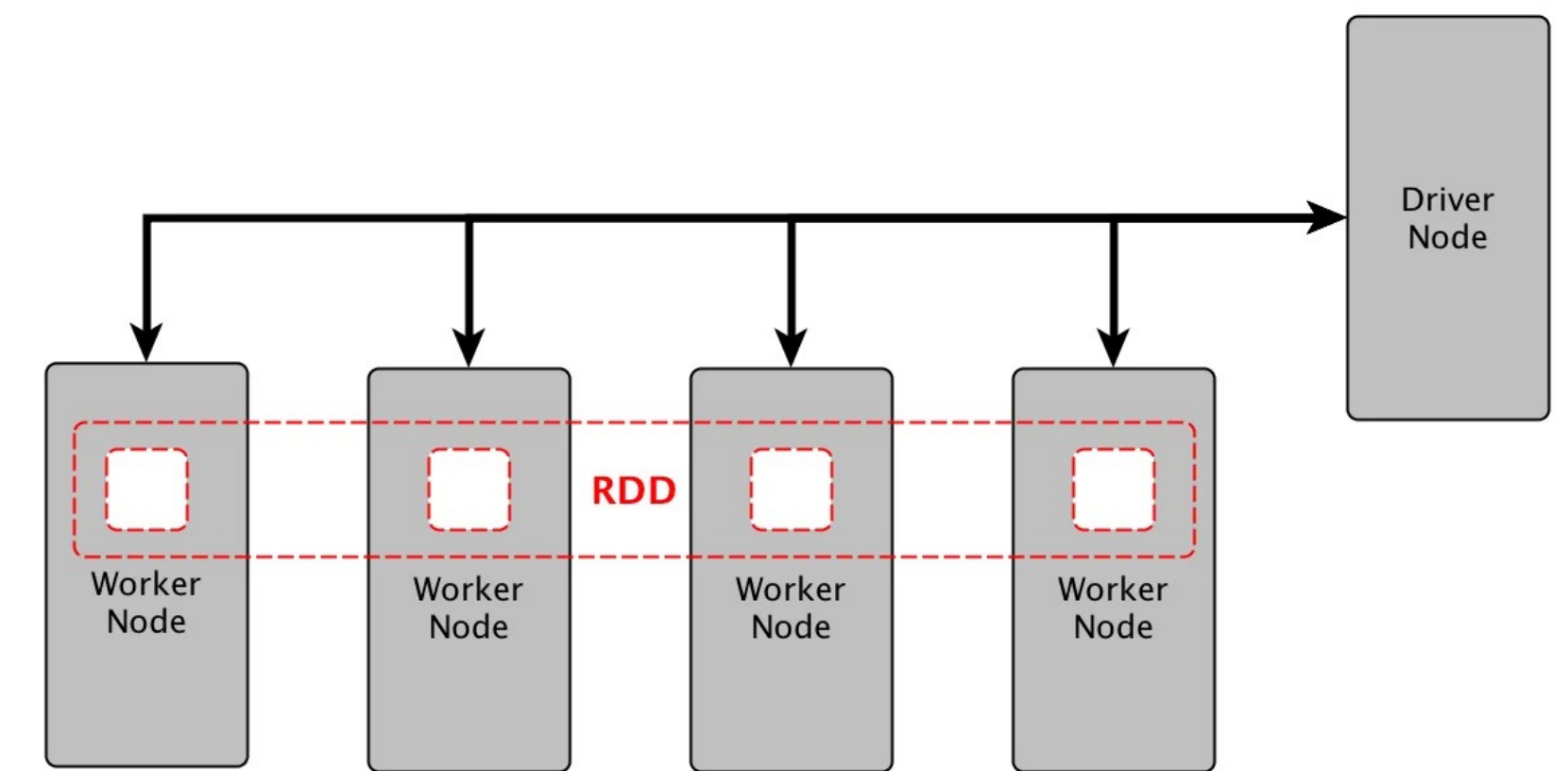
- Spark enables you to **dynamically adjust the resources** your application uses based on the **current workload**. This means that if your application no longer requires certain resources, it can return them to the cluster. Later, when demand increases, it can request those resources again. This feature is especially beneficial when multiple applications are sharing resources within your Spark cluster.

Executor

- **Worker processes** are responsible for executing tasks.
- Each executor:
 - Runs on a worker node.
 - Executes tasks on partitions of data.
 - Can cache data in memory/disk for reuse.
 - Reports progress and results back to the Driver.
- Executors are allocated based on available cluster resources.

RDD (Resilient Distributed Dataset)

- **Immutable** and **partitioned** across the cluster.
- Fault tolerance through lineage (recompute partitions when needed).
- Mostly replaced by DataFrames/Datasets for performance and ease of use.



https://miro.medium.com/v2/resize:fit:1400/format:webp/1*I2MUHFvWfcdiUbh7Y-fM5Q.png

Apache Spark

Execution Flow

3) Stage Division and Task Scheduling

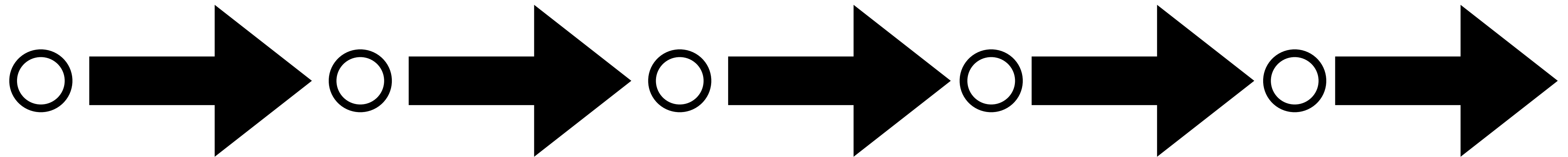
Job is split into stages (shuffle boundaries) and tasks (per partition).
Tasks are scheduled based on the optimised physical plan.

1) App Submission

User submits Spark application;
Driver creates SparkSession (and SparkContext)

5) Result Aggregation

Driver aggregates results, presents to user



2) Job Creation and DAG Creation

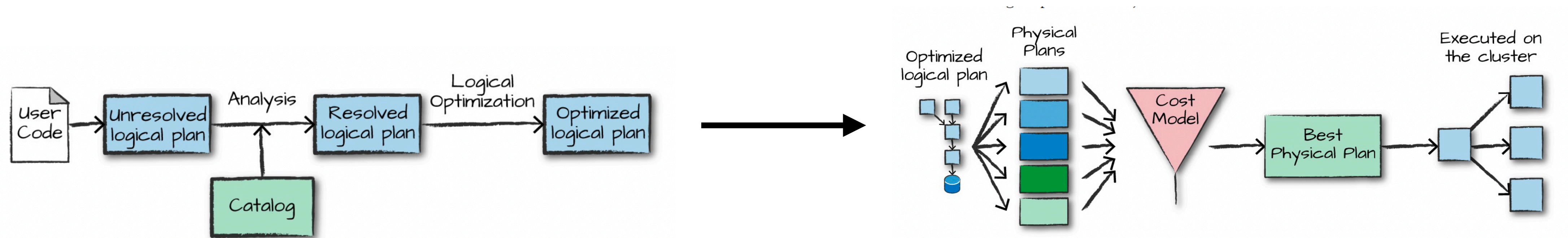
Driver converts code into jobs.
Transformations build a logical DAG (lazy plan).
An action triggers execution: Spark creates a job.

4) Task Execution on Worker Nodes

Tasks run on executors.
Results are returned or written to storage

Spark SQL and DataFrames

- DataFrames provide a high-level API for structured data.
- Both SQL and DataFrames use Catalyst Optimizer for efficient execution, namely for creating the optimised plan.
 - Catalyst optimizer rewrites queries for efficiency.
- Supports multiple data sources:
 - Parquet, ORC, JDBC, Hive, JSON.
- Typically faster and safer than raw RDD code for analytics.



Spark: The definitive Guide (Figure 4.2 and 4.3)

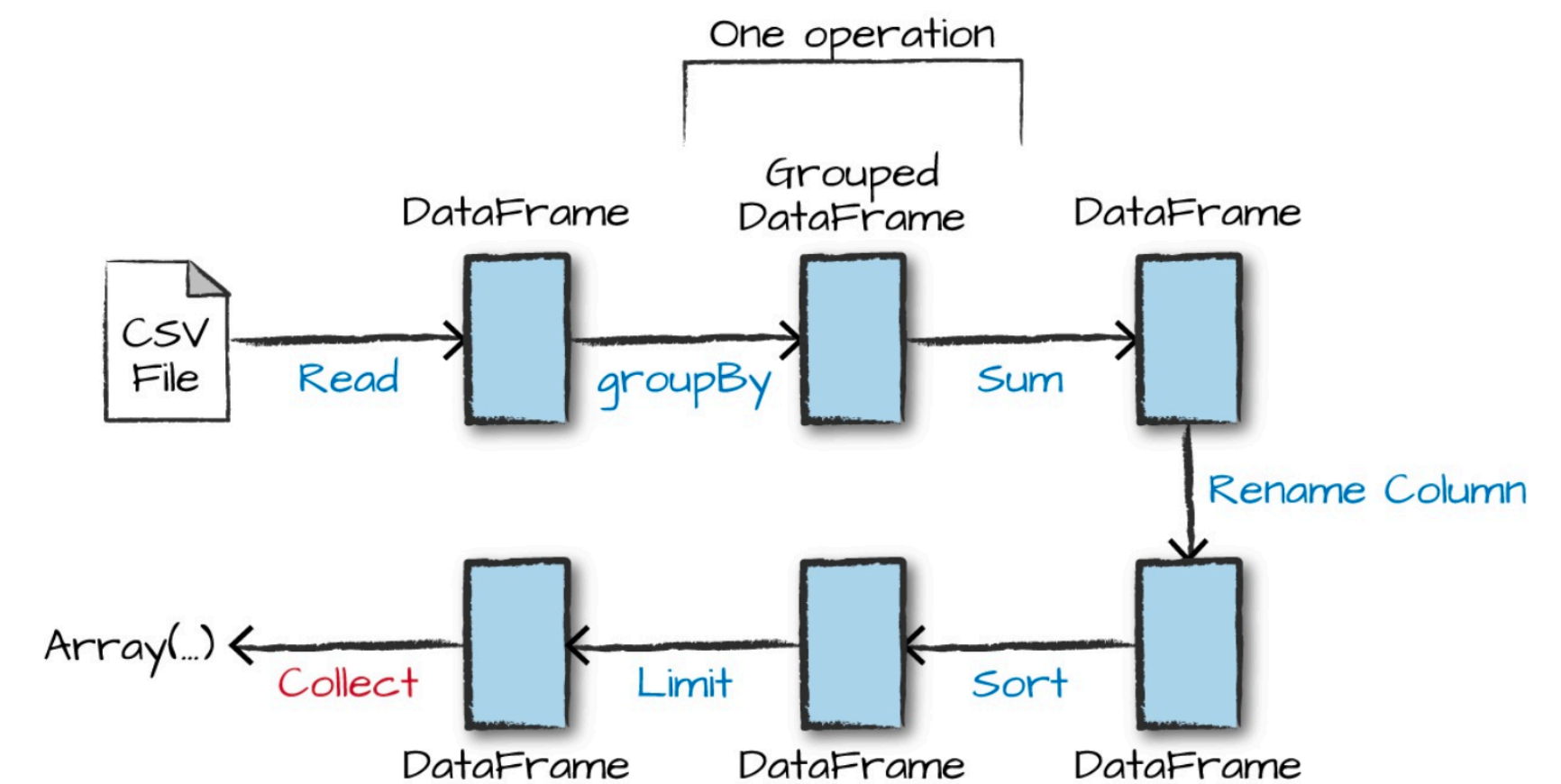
Spark Operations

● Apache Spark follows a **lazy evaluation model**.

- It records what to do, then runs when needed.

● Operations are divided into two types:

- **Transformations**, which define a new dataset (lazy).
- **Actions**, which trigger execution and return/write results.



Spark: The definitive Guide (Figure 2.10)

What are Transformations?

- **Transformations** produce a new dataset (RDD or Dataset) from an existing one.
- **Lazy Evaluation**: They do not execute immediately.
- Types:
 - **Narrow Transformations**: Data from each partition is used independently (e.g., ``map``, ``filter``, ``flatMap``).
 - **Wide Transformations**: Data from multiple partitions is shuffled (e.g., ``groupByKey``, ``reduceByKey``, ``sortByKey``, ``join``).
- **Note that** wide transformations usually define stage boundaries and cost more.

What are Actions?

- **Actions trigger execution** of the DAG and produce an output.
- Actions either:
 - Return a value to the Driver (e.g., ``count``, ``first``, ``take``).
 - Writes results to storage (e.g., ``write.parquet``, ``saveAsTextFile``).
- Examples:
 - ``count``, ``collect``, ``first``, ``take``, ``saveAs...``, ``reduce``, ``fold``.
- **Note that** ``collect()``, can overflow Driver memory!

Lazy Evaluation and DAG Execution

- Transformations are recorded but not executed until an action is called; this is based on Spark's **lazy evaluation**.
- Spark builds a **DAG** (execution plan) from your operations, which ultimately represents the execution flow.
- Before running, **Spark optimizes the plan** (pipelines where possible, reduces work).

Result: fewer passes over data and better performance!

Transformations vs. Actions Comparison

Feature	Transformations	Actions
Execution	Lazy, do not run a job	Triggers job execution
Output	New Dataset	Return a value or write output
Example	map, filter, reduceByKey	collect, count, first

Performance considerations

- **Bad configurations** lead to slow performance, excessive memory usage, and costly operations.

Performance considerations

Parallelism

- Two types of parallelism:
 - **Task Parallelism:** Tasks are divided and distributed across available **CPU cores**.
 - **Data Parallelism:** Data is partitioned across **multiple worker nodes** in a cluster.
- Single vs multi-core execution.

Performance considerations

Cache vs Persist

- Avoid recomputation of expensive transformations.
- **Cache:** Stores DataFrame in **memory only**.
 - **Use** `.cache()`
 - **when:** Dataset fits in memory and is used multiple times.
- **Persist:** Stores DataFrame with **custom storage levels**.
 - **Use** `.persist(StorageLevel.MEMORY_AND_DISK)`
 - **when:** Dataset is too large to fit in memory.

Performance considerations

Shuffle, Repartition and Coalesce

- Wide vs. Narrow Transformations
- **Narrow:** No shuffle (map, filter).

TRANSFORMATION	TYPE	SHUFFLE?	EXAMPLE
MAP(), FILTER()	NARROW	✗ NO SHUFFLE	DF.FILTER(DF.YEAR > 2000)
GROUPBY(), JOIN()	WIDE	✓ CAUSES SHUFFLE	DF.GROUPBY("GENRE").COUNT()

Performance considerations

Shuffle, Repartition and Coalesce

- **Repartition (`df.repartition(10)`):** Increases partitions and triggers shuffle.
 - **When:** increasing parallelism.
- **Coalesce (`df.coalesce(4)`):** Reduces partitions **without shuffling**.
 - **When:** reducing partitions after a shuffle.

Performance considerations

Joins and Broadcast

- **Shuffle Join** happens when joining **large** DataFrames.
- **Broadcast Join** improves performance by sending small DataFrame to all executors.

Performance considerations

Execution Plans

- Why lazy evolution works!
- **`explain("simple")`**: Overview of the execution plan.
- **`explain("extended")`**: Full plan with shuffles, exchanges, and stage details.

Performance considerations

Efficient I/O in Spark

- Use **Parquet** format instead of CSV for faster reads/writes.
 - `df.write.parquet()`
- Enable **compression** (e.g., gzip, snappy) to reduce storage size.
 - ❖ `df.write.option("compression", "gzip")`
- Partitioning improves query speed.
 - `df.write.partitionBy("startYear")`

Performance Considerations (Summary)

- Prefer **reduceByKey()** over **groupByKey()** to minimize shuffling.
 - It reduces shuffle volume by combining before transfer.
- Use **persist()** or **cache()** for reuse of datasets.
 - Reusing the same dataset across multiple actions is a common way to perform things. Then, you should use these configurations.
- Avoid **collect()** on large datasets.
 - It moves all data to the Driver memory!
- Use **repartition()/coalesce()** wisely to optimize parallelism.
 - These configurations should be used to balance parallelism vs shuffle cost.

Checkpointing and Fault Tolerance

- **Checkpointing** stores state/progress information to recover after failures.
- Stores metadata/state in reliable storage (e.g., HDFS/S3).
- On restart, Spark resumes from the last committed checkpoint.

Know more about

- **Glossary:** <https://spark.apache.org/docs/latest/cluster-overview.html>
- **Scheduling and dynamic resource allocation:** <https://spark.apache.org/docs/latest/job-scheduling.html>
- **Overall information:**
 - Spark: The Definitive Guide. Bill Chambers, Matei Zaharia. 2018. O' Reilly Media, Inc
 - Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Martin Kleppmann. 2017. O' Reilly Media, Inc
- **Tuning and Optimization:** <http://spark.apache.org/docs/latest/tuning.html>

Questions?